

★ Member-only story

7 Python Libraries I Install Before I Start Any Project

They make everything feel easier from day one



Hassan Nauman

Follow

4 min read · Jan 21, 2026



245



1





Generated by Sora

I assume you've been writing Python long enough to be dangerous.

You know the syntax. You've shipped code. You've debugged things at 2 a.m. that *definitely* worked at 6 p.m.

Yet every new project still starts the same way:
a blank folder, a virtual environment, and that quiet thought — *"There has to be a smarter way to do this."*

After 4+ years of writing Python professionally, here's one thing I've learned the hard way:

The libraries you install in the first 10 minutes decide how painful the next 3 months will be.

These are the 7 Python libraries I install before I write a single line of business logic.

Not trendy. Not overhyped. Just unfairly powerful.

1. `rich` — Because Print Statements Deserve Respect

If you're still using plain `print()` for debugging, you're living in the Stone Age.

`rich` turns your terminal into a developer-grade dashboard.

```
from rich import print
from rich.console import Console
from rich.table import Table

console = Console()
table = Table(title="User Stats")
table.add_column("Name", style="cyan")
table.add_column("Score", justify="right", style="green")
table.add_row("Alice", "98")
table.add_row("Bob", "87")
console.print(table)
```

Why this matters:

- Readable logs reduce debugging time (by a lot)
- Structured output beats scrolling through text soup
- You can visualize JSON, tracebacks, progress bars, and trees

Developers spend ~35–40% of their time debugging. `rich` makes that time cheaper.

Once you use it, plain `print()` feels like shouting into the void.

2. pydantic — Make Bad Data Someone Else's Problem

Every bug eventually traces back to **bad input**.

pydantic enforces structure *before* chaos enters your system.

```
from pydantic import BaseModel, ValidationError

class User(BaseModel):
    id: int
    email: str
    is_active: bool = True
try:
    user = User(id="abc", email="not-an-email")
except ValidationError as e:
    print(e)
```

Why I install it immediately:

- Automatic type validation
- Self-documenting models
- Fail fast, fail loud

Pydantic is the reason FastAPI feels magical — but it's just as powerful in scripts, data pipelines, and CLIs.

If your project touches APIs, configs, or user input, this is non-negotiable.

3. python-dotenv — Because Secrets Don't Belong in Code

Every beginner hardcodes API keys.

Every expert regrets it.

```
# .env
DATABASE_URL=postgresql://user:pass@localhost/db
API_KEY=super_secret_key
```

```
from dotenv import load_dotenv
import os

load_dotenv()
db_url = os.getenv("DATABASE_URL")
api_key = os.getenv("API_KEY")
```

Why this library quietly saves careers:

- Keeps secrets out of Git
- Makes deployments predictable
- Works everywhere (local, Docker, CI)

Real talk: Leaked credentials are still one of the top causes of production incidents.

This library prevents that with almost zero effort.

4. typer — CLI Tools Without the Pain

Every serious project eventually needs a command-line interface.

argparse works.

typer feels like cheating.

```
import typer

app = typer.Typer()
@app.command()
def greet(name: str, admin: bool = False):
    if admin:
        typer.echo(f"Welcome back, admin {name}")
    else:
        typer.echo(f"Hello {name}")
if __name__ == "__main__":
    app()
```

Run:

```
python app.py greet Alice --admin
```

Why typer is underrated:

- Auto-generated help docs
- Type hints become CLI validation
- Built on `click` (battle-tested)

[Open in app](#) ↗



Medium



Search



Write



5. loguru — Logging That Doesn't Hate You Back

Python's built-in logging module is... an experience.

loguru is what logging should have been from day one.

```
from loguru import logger

logger.add("app.log", rotation="1 MB")
logger.info("Application started")
logger.warning("Something feels off")
logger.error("Something broke")
```

Why this matters more than you think:

- Zero configuration
- Structured logs by default
- Rotating files without ceremony

*Good logs don't help when things work.
They save you when everything is on fire.*

6. tenacity — Because Networks Lie

APIs fail.

Databases blink.

Networks have moods.

tenacity gives you **retries with dignity**.

```
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(stop=stop_after_attempt(3), wait=wait_exponential())
def fetch_data():
    print("Trying...")
    raise Exception("Temporary failure")
fetch_data()
```

Why I install it early:

- Eliminates flaky behavior
- Makes failures predictable
- Saves you from writing retry spaghetti

If your project touches anything over the network, you need this.

7. watchdog — React to File Changes Like a Pro

This one is criminally underused.

watchdog lets your program react to file system events in real time.

```
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
import time

class Handler(FileSystemEventHandler):
    def on_modified(self, event):
        print(f"{event.src_path} changed")
observer = Observer()
observer.schedule(Handler(), path=".", recursive=True)
observer.start()
try:
```

```
while True:
    time.sleep(1)
except KeyboardInterrupt:
    observer.stop()
observer.join()
```

Use cases:

- Auto-reload configs
- Trigger pipelines on file drops
- Build dev tools without polling

Once you've built one reactive system, polling feels primitive.

Final Thought

Most developers obsess over frameworks.

Experts obsess over **foundations**.

These libraries don't just save time —
they **change how your code behaves under pressure**.

If I install these seven first, I know one thing for sure:

The project will age better than my patience.

If you want the next article, I'll break down **the 5 libraries I remove once the project hits production**.

Trust me — that list is just as important.

Enjoyed this one? Show some love with 50 claps 🖐️ and hit Follow to stay tuned for upcoming posts packed with fresh perspectives.
Appreciate your time — see you in the next article! 🌟
Thanks a lot for reading! 🙌

- Python
- Programming
- Data Science
- Artificial Intelligence
- Technology



Published in Python in Plain English

Following

93K followers · Last published just now

New Python content every day. Follow to join our 3.5M+ monthly readers.



Written by Hassan Nauman

Follow

690 followers · 7 following

Computer Engineer | Writer | C++ & Python Developer | Freelancer | Blogger | Tech Explorer

Responses (1)



Tony Zoppi

What are your thoughts?





Tom Herden
4 days ago



Interesting collection, **but**: your example code for `rich` does nothing to illustrate how to use it to create "readable logs", let alone a "developer-grade console".



[Reply](#).

More from Hassan Nauman and Python in Plain English



Automation Systems



In Stackademic by Hassan Nauman

5 Automation Systems Every Developer Should Set Up for...

Less routine, more deep work



In Python in Plain English by Its Shoryabisht

Pandas 3.0: New Features That Change Python Data Analysis...

Pandas 3.0: The Game-Changing Evolution That Reshapes Python Data Analysis



Nov 28, 2025



519



9



Feb 1

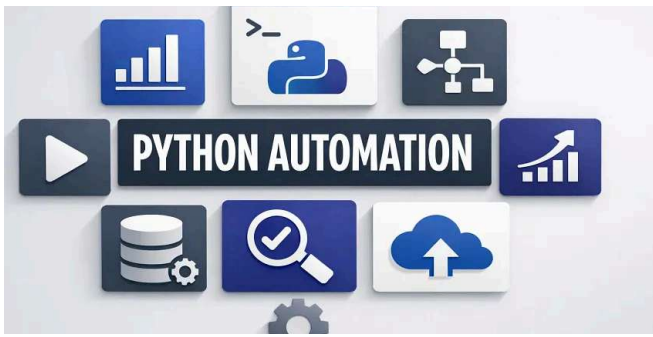


334



6





PY In Python in Plain English by Abdur Rahman

9 Python Libraries That Make Automation Ridiculously Easy

I automate first, ask questions later.

★ Feb 3 🖱 567 💬 8



In Stackademic by Hassan Nauman

How Developers Actually Make Money Outside Salaries

Honest paths that don't involve "gurus"

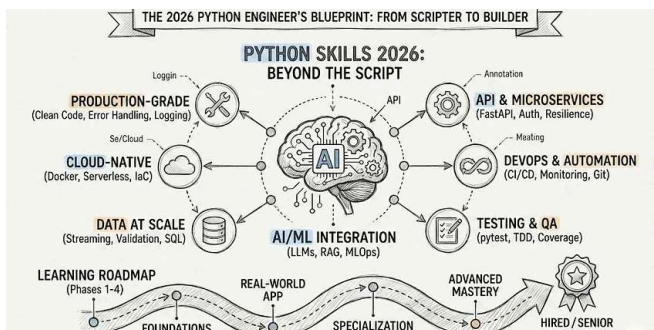
★ Dec 29, 2025 🖱 506 💬 10



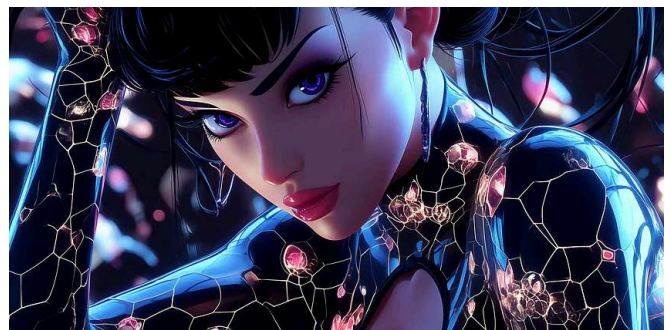
See all from Hassan Nauman

See all from Python in Plain English

Recommended from Medium



PY In Python in Plain English by Faisal haque

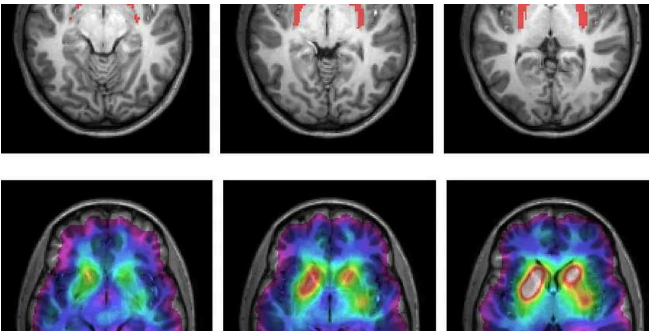


In AI Advances by Delanoe Pirard

The Python Skills That Actually Get You Hired in 2026 (And How to...

Why “Knowing Python” Isn’t Enough Anymore

Feb 8 394 14

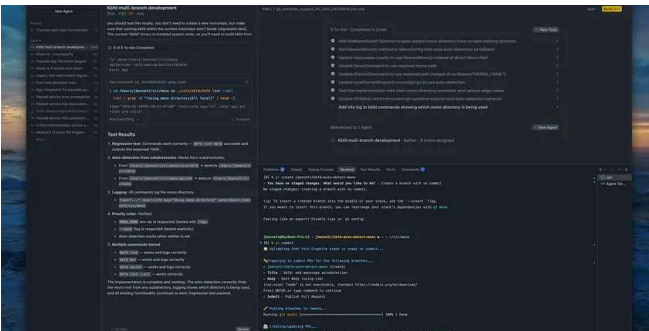


In Write A Catalyst by Dr. Patricia Schmidt

As a Neuroscientist, I Quit These 5 Morning Habits That Destroy You...

Most people do #1 within 10 minutes of waking (and it sabotages your entire day)

Jan 14 29K 503



Jacob Bennett

The 5 paid subscriptions I actually use in 2026 as a Staff Software...

Tools I use that are (usually) cheaper than Netflix

A 1967 Math Paper Just Solved AI’s \$100 Million Problem

December 31st, 2025. While Silicon Valley popped champagne, 20 researchers in...

Jan 28 1.2K 19



In AI Software Engineer by Joe Njenga

Anthropic Just Released Claude Code Course (And I Earned My...

Anthropic just launched their Claude Code in Action course, and I’ve just passed—how...

Jan 21 2.4K 35



In The Pythonworld by Aashish Kumar

Stop Writing Utility Functions in Python (Here’s the Better Pattern)

Utility functions feel productive—until they quietly turn your clean codebase into a...

★ Jan 18 🖱 2.7K 💬 68



★ Jan 23 🖱 601 💬 11



See more recommendations